

Exploration in RL: Contextual Bandit

Kianté Brantley
CS 1840 Fall 2026

Announcements

HW2 is out!

My office hours this week are < ... >.

Check the course website for Office Hours:

<https://harvard-cs-1840.github.io/2026/>

Finishing Up From Last Lecture

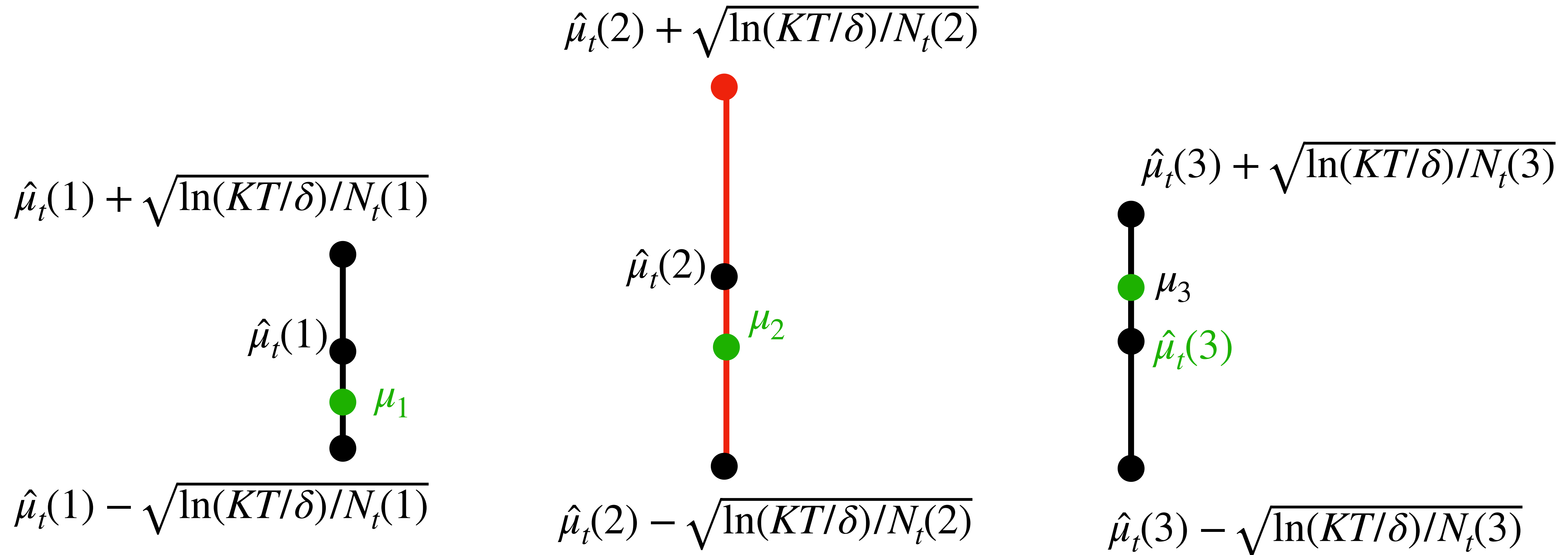


1. The upper Confidence Bound Algorithm

2. Analysis of UCB algorithm

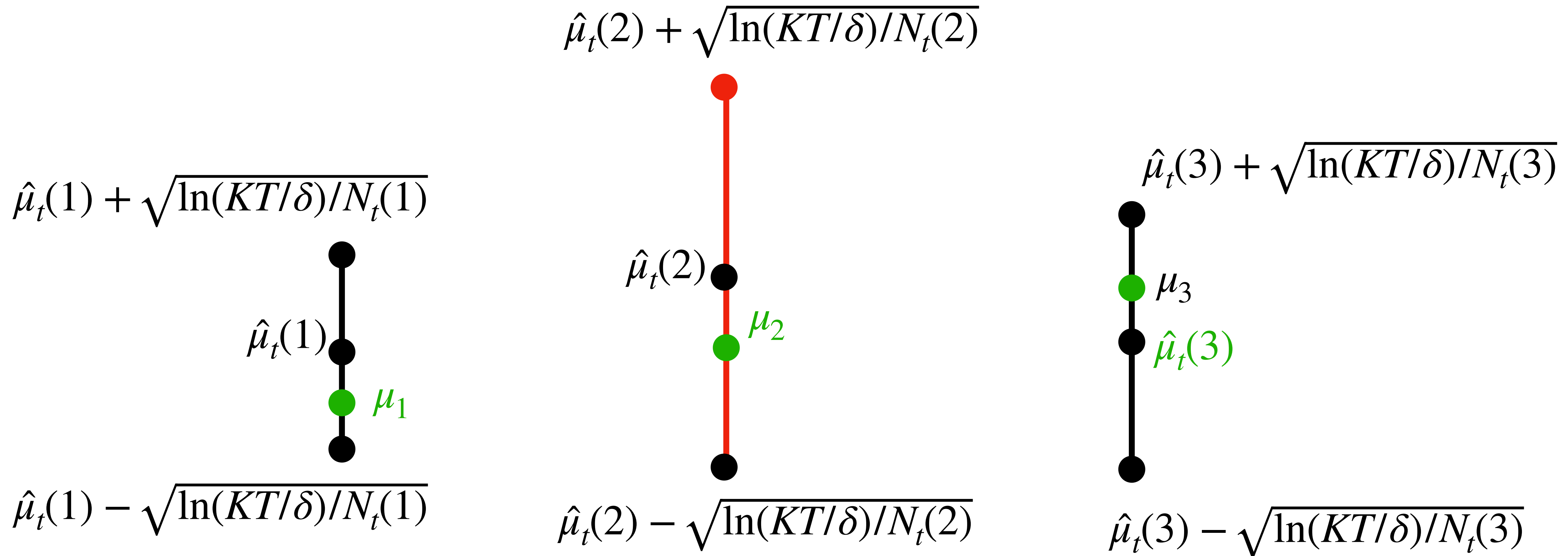
Recap: Intuitive Explanation of UCB

Recap: Intuitive Explanation of UCB



Recap: Intuitive Explanation of UCB

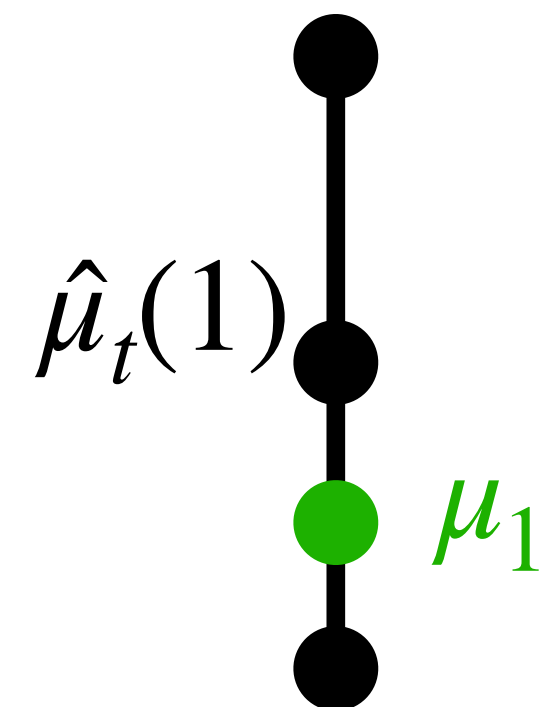
Case 1: it has large conf-interval, which means that it has not been tried many times yet (high uncertainty)



Recap: Intuitive Explanation of UCB

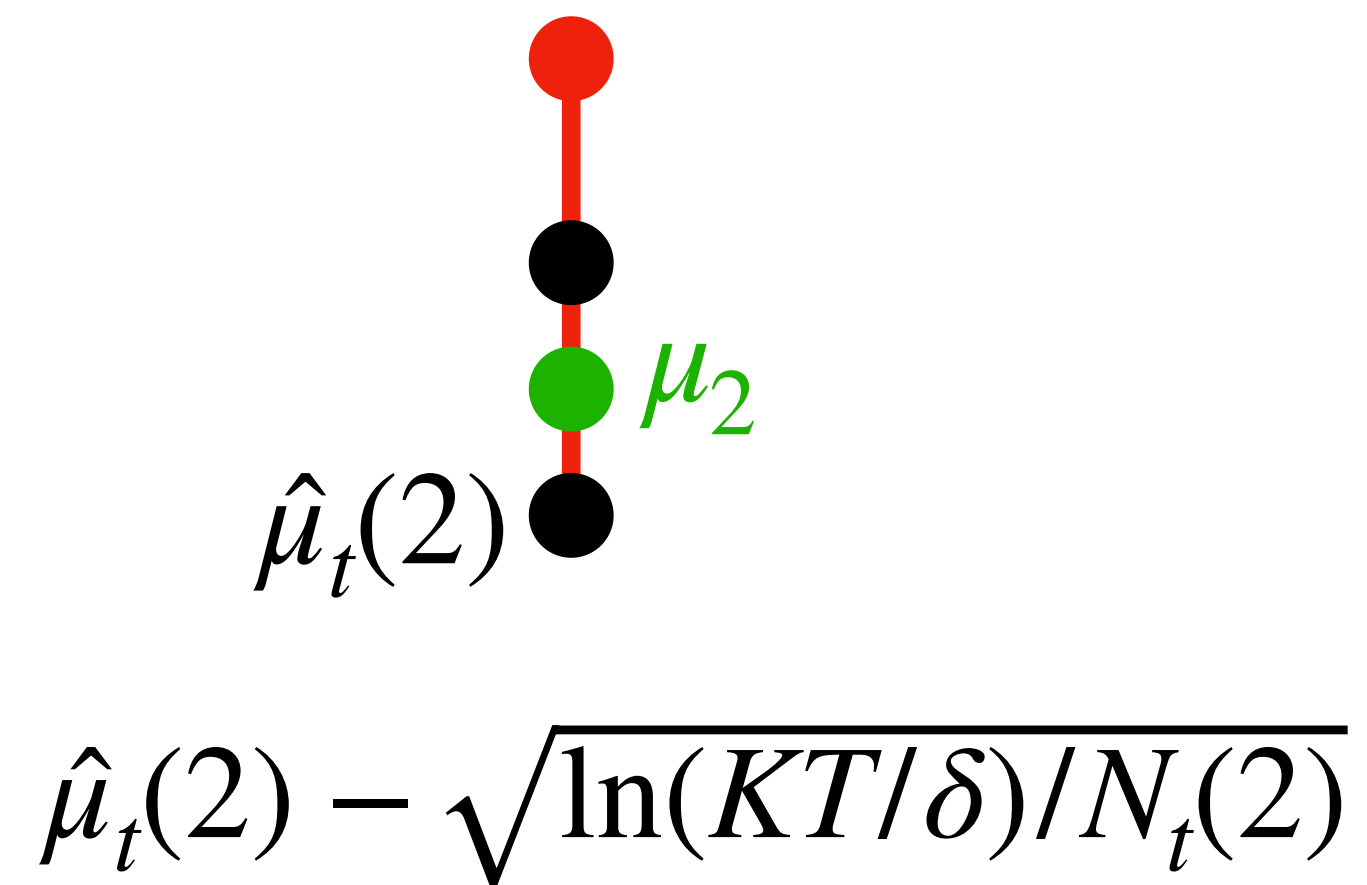
Recap: Intuitive Explanation of UCB

$$\hat{\mu}_t(1) + \sqrt{\ln(KT/\delta)/N_t(1)}$$



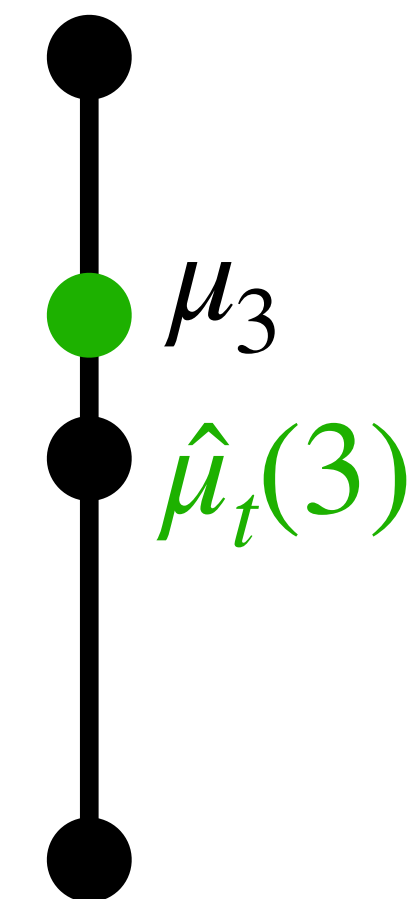
$$\hat{\mu}_t(1) - \sqrt{\ln(KT/\delta)/N_t(1)}$$

$$\hat{\mu}_t(2) + \sqrt{\ln(KT/\delta)/N_t(2)}$$



$$\hat{\mu}_t(2) - \sqrt{\ln(KT/\delta)/N_t(2)}$$

$$\hat{\mu}_t(3) + \sqrt{\ln(KT/\delta)/N_t(3)}$$



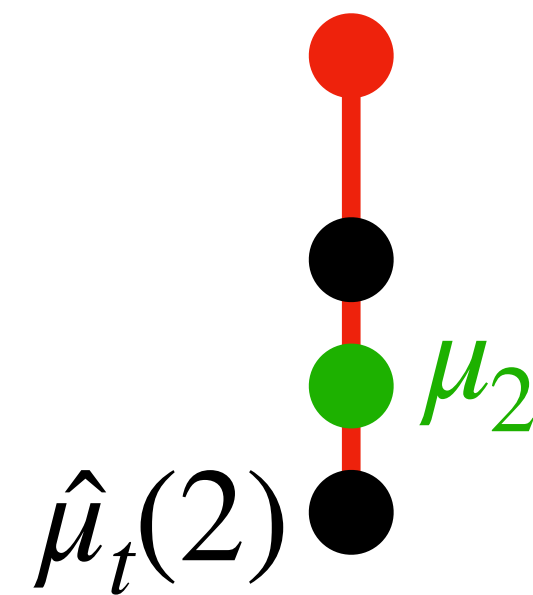
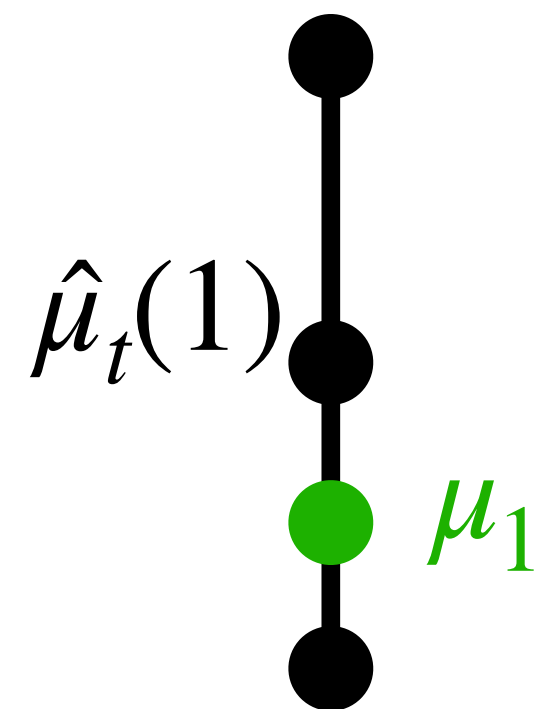
$$\hat{\mu}_t(3) - \sqrt{\ln(KT/\delta)/N_t(3)}$$

Recap: Intuitive Explanation of UCB

Case 2: it has low uncertainty, then it is simply a good arm, i.e., it's true mean is high!

$$\hat{\mu}_t(2) + \sqrt{\ln(KT/\delta)/N_t(2)}$$

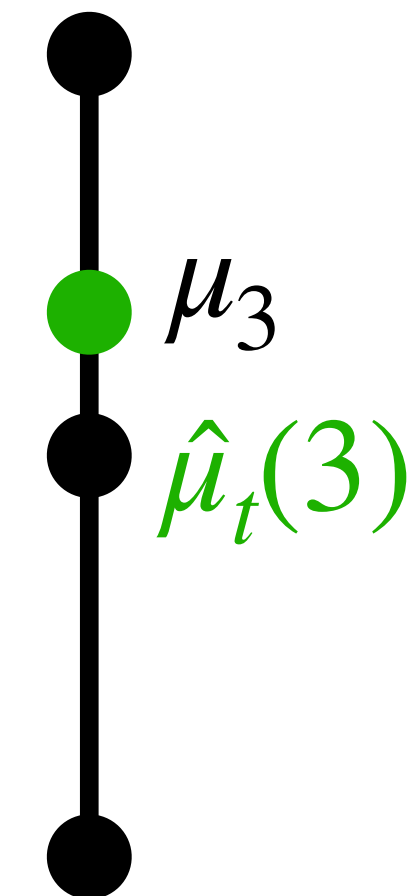
$$\hat{\mu}_t(1) + \sqrt{\ln(KT/\delta)/N_t(1)}$$



$$\hat{\mu}_t(2) - \sqrt{\ln(KT/\delta)/N_t(2)}$$

$$\hat{\mu}_t(1) - \sqrt{\ln(KT/\delta)/N_t(1)}$$

$$\hat{\mu}_t(3) + \sqrt{\ln(KT/\delta)/N_t(3)}$$



$$\hat{\mu}_t(3) - \sqrt{\ln(KT/\delta)/N_t(3)}$$

Recap: Exploration–Exploitation Tradeoff

Case 1: a_t has large conf-interval, which means that it has not been tried many times yet (high uncertainty)

Thus, we do exploration in this case!

Recap: Exploration–Exploitation Tradeoff

Case 1: a_t has large conf-interval, which means that it has not been tried many times yet (high uncertainty)

Thus, we do exploration in this case!

Case 2: a_t has small conf-interval, then it is simply a good arm, i.e., its true mean is pretty high!

Thus, we do exploitation in this case!

Let's formalize the intuition

Denote the optimal arm $k^\star = \arg \max_{i \in [K]} \mu_i$; recall $a_t = \arg \max_{i \in [K]} \hat{\mu}_t(i) + \sqrt{\frac{\ln(KT/\delta)}{N_t(i)}}$

Let's formalize the intuition

Denote the optimal arm $k^\star = \arg \max_{i \in [K]} \mu_i$; recall $a_t = \arg \max_{i \in [K]} \hat{\mu}_t(i) + \sqrt{\frac{\ln(KT/\delta)}{N_t(i)}}$

$$\text{Regret-at-}t = \mu^\star - \mu_{a_t}$$

Let's formalize the intuition

Denote the optimal arm $k^\star = \arg \max_{i \in [K]} \mu_i$; recall $a_t = \arg \max_{i \in [K]} \hat{\mu}_t(i) + \sqrt{\frac{\ln(KT/\delta)}{N_t(i)}}$

$$\text{Regret-at-}t = \mu^\star - \mu_{a_t}$$

Q: why?

$$\leq \hat{\mu}_t(a_t) + \sqrt{\frac{\ln(TK/\delta)}{N_t(a_t)}} - \mu_{a_t}$$

Let's formalize the intuition

Denote the optimal arm $k^\star = \arg \max_{i \in [K]} \mu_i$; recall $a_t = \arg \max_{i \in [K]} \hat{\mu}_t(i) + \sqrt{\frac{\ln(KT/\delta)}{N_t(i)}}$

$$\begin{aligned} \text{Regret-at-}t &= \mu^\star - \mu_{a_t} \\ &\leq \hat{\mu}_t(a_t) + \sqrt{\frac{\ln(TK/\delta)}{N_t(a_t)}} - \mu_{a_t} \end{aligned}$$

Let's formalize the intuition

Denote the optimal arm $k^\star = \arg \max_{i \in [K]} \mu_i$; recall $a_t = \arg \max_{i \in [K]} \hat{\mu}_t(i) + \sqrt{\frac{\ln(KT/\delta)}{N_t(i)}}$

$$\begin{aligned} \text{Regret-at-}t &= \mu^\star - \mu_{a_t} \\ &\leq \hat{\mu}_t(a_t) + \sqrt{\frac{\ln(TK/\delta)}{N_t(a_t)}} - \mu_{a_t} \\ &\leq 2\sqrt{\frac{\ln(TK/\delta)}{N_t(a_t)}} \end{aligned}$$

Let's formalize the intuition

Denote the optimal arm $k^\star = \arg \max_{i \in [K]} \mu_i$; recall $a_t = \arg \max_{i \in [K]} \hat{\mu}_t(i) + \sqrt{\frac{\ln(KT/\delta)}{N_t(i)}}$

$$\begin{aligned} \text{Regret-at-}t &= \mu^\star - \mu_{a_t} \\ &\leq \hat{\mu}_t(a_t) + \sqrt{\frac{\ln(TK/\delta)}{N_t(a_t)}} - \mu_{a_t} \\ &\leq 2\sqrt{\frac{\ln(TK/\delta)}{N_t(a_t)}} \end{aligned}$$

Case 1: $N_t(a_t)$ is small
(i.e., uncertainty about a_t is large);

We pay regret, BUT we **explore** here,
as we just tried a_t at iter t !

Let's formalize the intuition

Denote the optimal arm $k^\star = \arg \max_{i \in [K]} \mu_i$; recall $a_t = \arg \max_{i \in [K]} \hat{\mu}_t(i) + \sqrt{\frac{\ln(KT/\delta)}{N_t(i)}}$

$$\begin{aligned} \text{Regret-at-}t &= \mu^\star - \mu_{a_t} \\ &\leq \hat{\mu}_t(a_t) + \sqrt{\frac{\ln(TK/\delta)}{N_t(a_t)}} - \mu_{a_t} \\ &\leq 2\sqrt{\frac{\ln(TK/\delta)}{N_t(a_t)}} \end{aligned}$$

Case 2: $N_t(a_t)$ is large, i.e., conf-interval of a_t is small,

Then we **exploit** here, as a_t is pretty good (the gap between $\mu^\star$ & μ_{a_t} is small)!

Let's formalize the intuition

Finally, let's add all per-iter regret together:

$$\begin{aligned}\text{Regret}_T &= \sum_{t=0}^{T-1} \left(\mu^\star - \mu_{a_t} \right) \\ &\leq \sum_{t=0}^{T-1} 2\sqrt{\frac{\ln(TK/\delta)}{N_t(a_t)}} \\ &= 2\sqrt{\ln(TK/\delta)} \cdot \sum_{t=0}^{T-1} \sqrt{\frac{1}{N_t(a_t)}}\end{aligned}$$

Let's formalize the intuition

Finally, let's add all per-iter regret together:

$$\begin{aligned}\text{Regret}_T &= \sum_{t=0}^{T-1} \left(\mu^\star - \mu_{a_t} \right) \\ &\leq \sum_{t=0}^{T-1} 2 \sqrt{\frac{\ln(TK/\delta)}{N_t(a_t)}} \\ &= 2\sqrt{\ln(TK/\delta)} \cdot \sum_{t=0}^{T-1} \sqrt{\frac{1}{N_t(a_t)}}\end{aligned}$$

Lemma (optional):

$$\sum_{t=0}^{T-1} \sqrt{\frac{1}{N_t(a_t)}} \leq O\left(\sqrt{KT}\right)$$

UCB Regret:

[Theorem (informal)] With high probability, UCB has the following regret:

$$\text{Regret}_T = \widetilde{O} \left(\sqrt{KT} \right)$$

(See reading material for more details)

Recap: MAB

Interactive learning process:

For $t = 0 \rightarrow T - 1$

(# based on historical information)

1. Learner pulls arm $a_t \in \{1, \dots, K\}$
2. Learner observes an i.i.d reward $r_t \sim \nu_{a_t}$ of arm a_t

Recap: MAB

Interactive learning process:

For $t = 0 \rightarrow T - 1$

(# based on historical information)

1. Learner pulls arm $a_t \in \{1, \dots, K\}$
2. Learner observes an i.i.d reward $r_t \sim \nu_{a_t}$ of arm a_t

Learning metric:

$$\text{Regret}_T = T\mu^\star - \sum_{t=0}^{T-1} \mu_{a_t}$$

Recap: MAB

Interactive learning process:

For $t = 0 \rightarrow T - 1$

(# based on historical information)

1. Learner pulls arm $a_t \in \{1, \dots, K\}$
2. Learner observes an i.i.d reward $r_t \sim \nu_{a_t}$ of arm a_t

Learning metric:

$$\text{Regret}_T = T\mu^\star - \sum_{t=0}^{T-1} \mu_{a_t}$$

Arm distributions are fixed across learning..

Question for Today:

How to incorporate contexts into the
interactive learning framework

Outline for today:

1. Introduction of the model

2. Algorithm

3. Theory and some practical considerations

Make the framework Context Dependent:

Interactive learning process:

For $t = 0 \rightarrow T - 1$

1. A new context $x_t \in \mathcal{X}$ appears

Make the framework Context Dependent:

Interactive learning process:

For $t = 0 \rightarrow T - 1$

1. A new context $x_t \in \mathcal{X}$ appears

2. Learner picks action $a_t \in \mathcal{A}$

(# based on context x_t and
historical information)

Make the framework Context Dependent:

Interactive learning process:

For $t = 0 \rightarrow T - 1$

1. A new context $x_t \in \mathcal{X}$ appears

(# based on context x_t and
historical information)

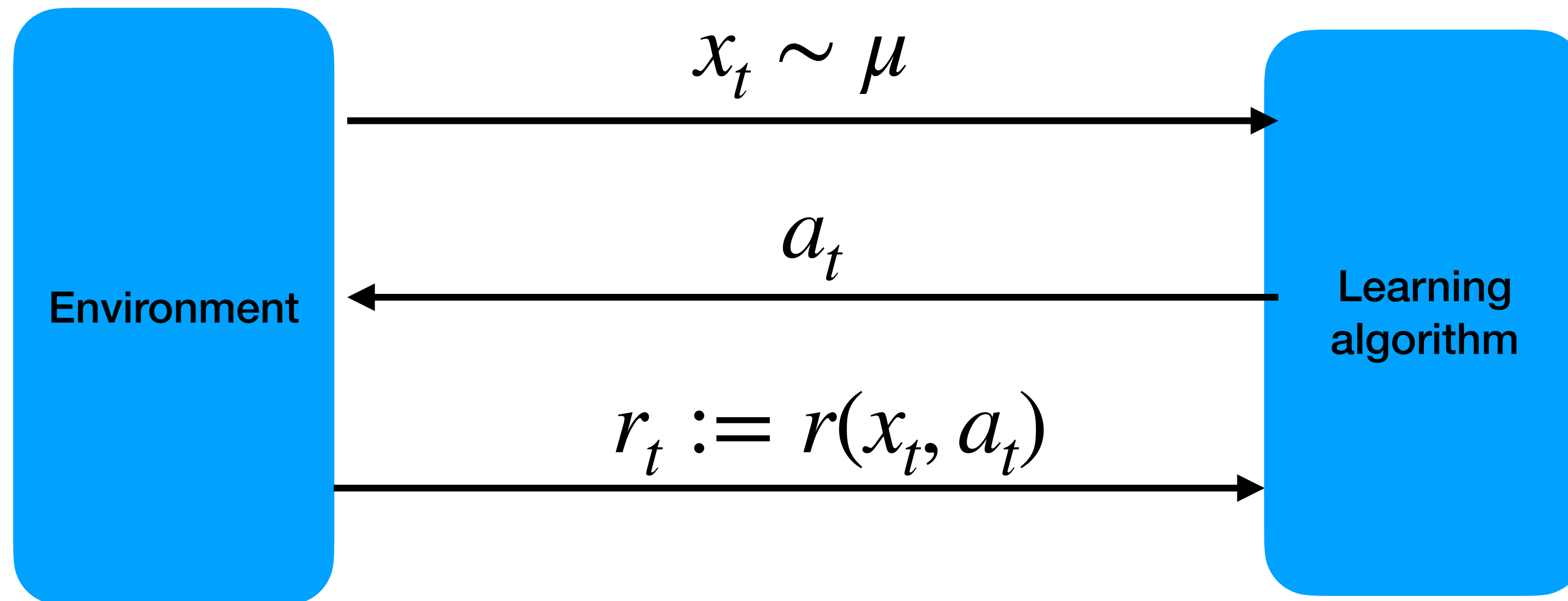
2. Learner picks action $a_t \in \mathcal{A}$

3. Learner observes an reward $r_t := r(x_t, a_t)$

Reward is context and arm
dependent now!

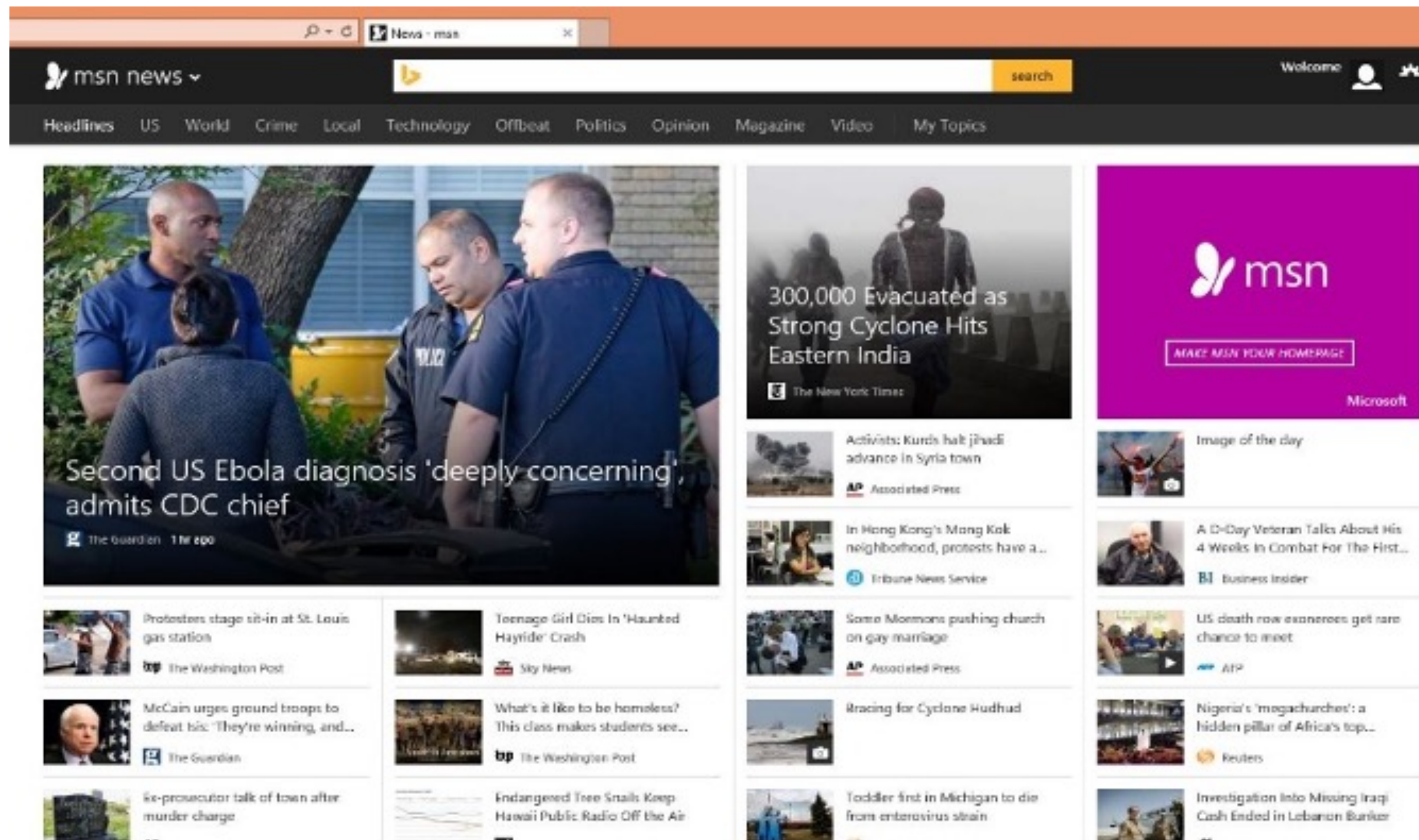
Make the framework Context Dependent:

Interactive learning process:



Examples:

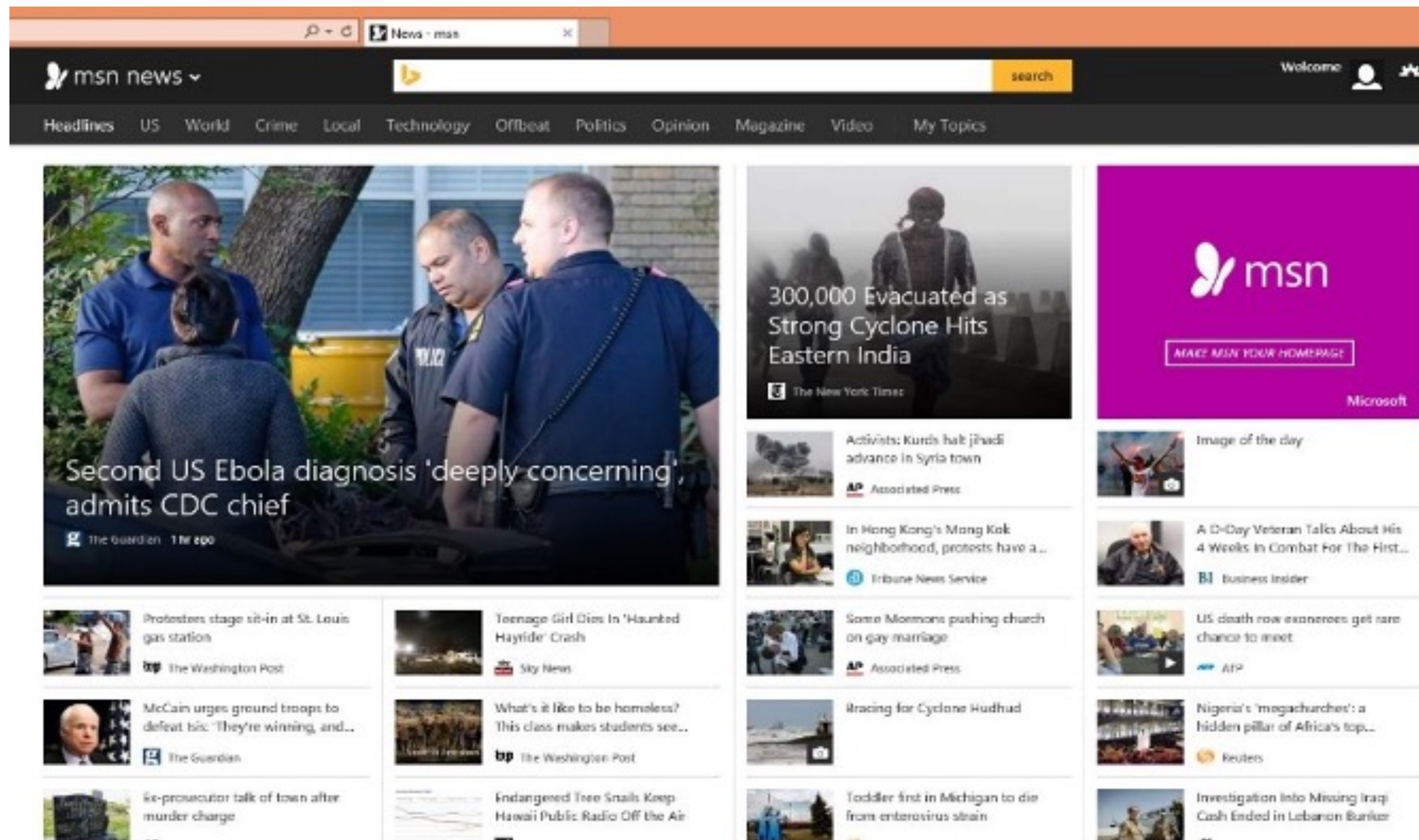
Personalize
recommendation system



Examples:

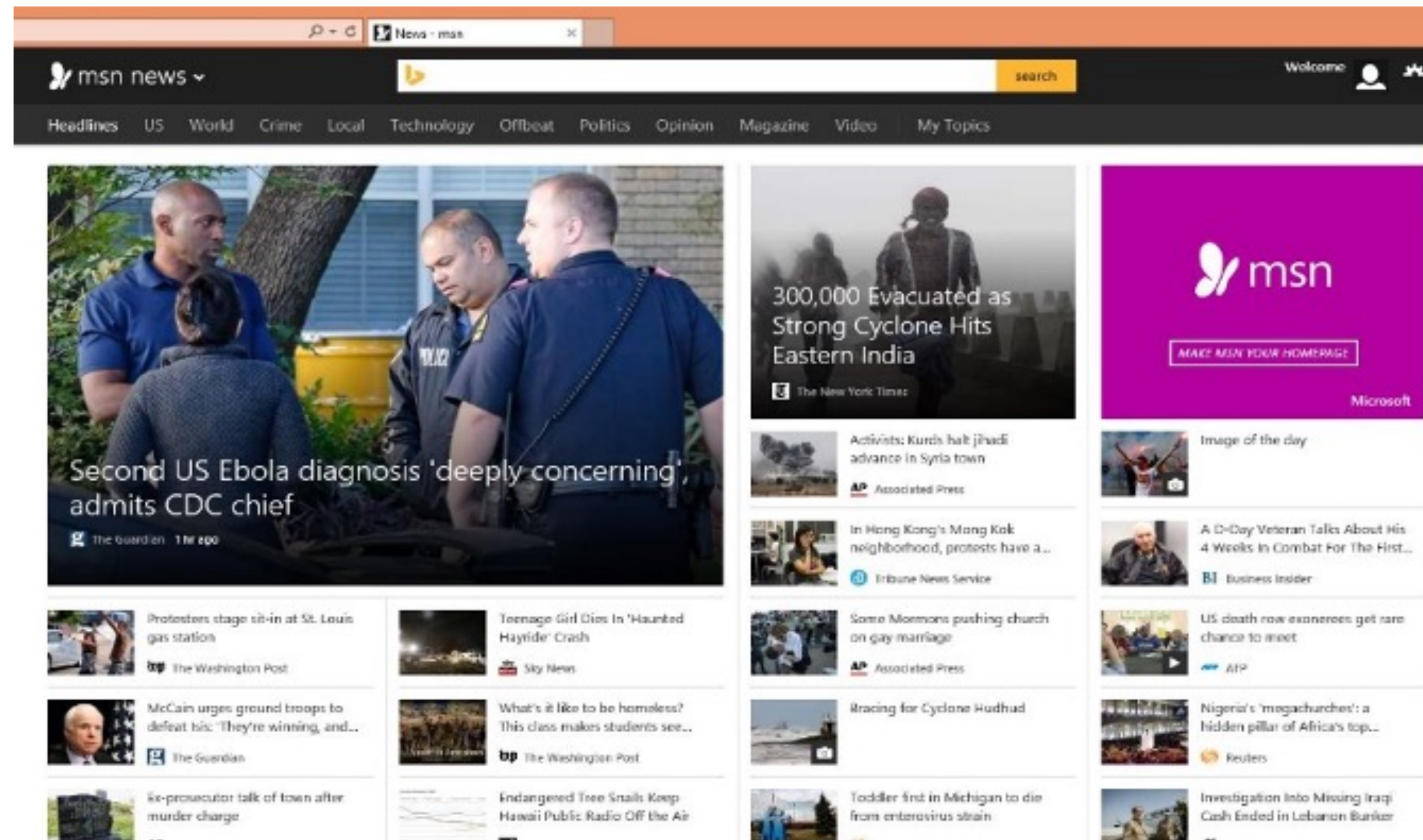
Personalize
recommendation system

Context: user's information (e.g.,
history health conditions, age, height,
weight, job type, etc)



Examples:

Personalize
recommendation system

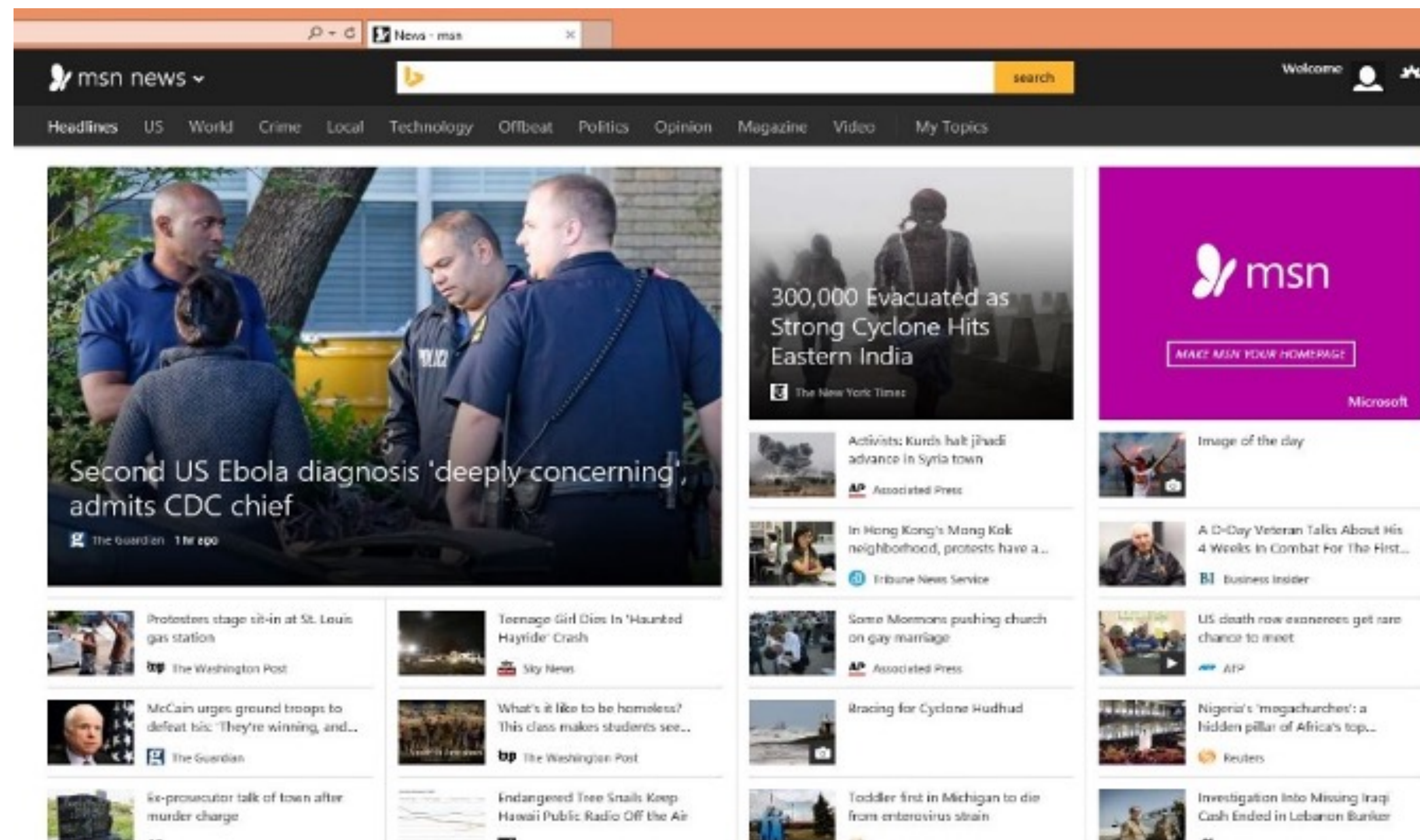


Context: user's information (e.g.,
history health conditions, age, height,
weight, job type, etc)

Decisions (arms): news articles

Examples:

Personalize
recommendation system



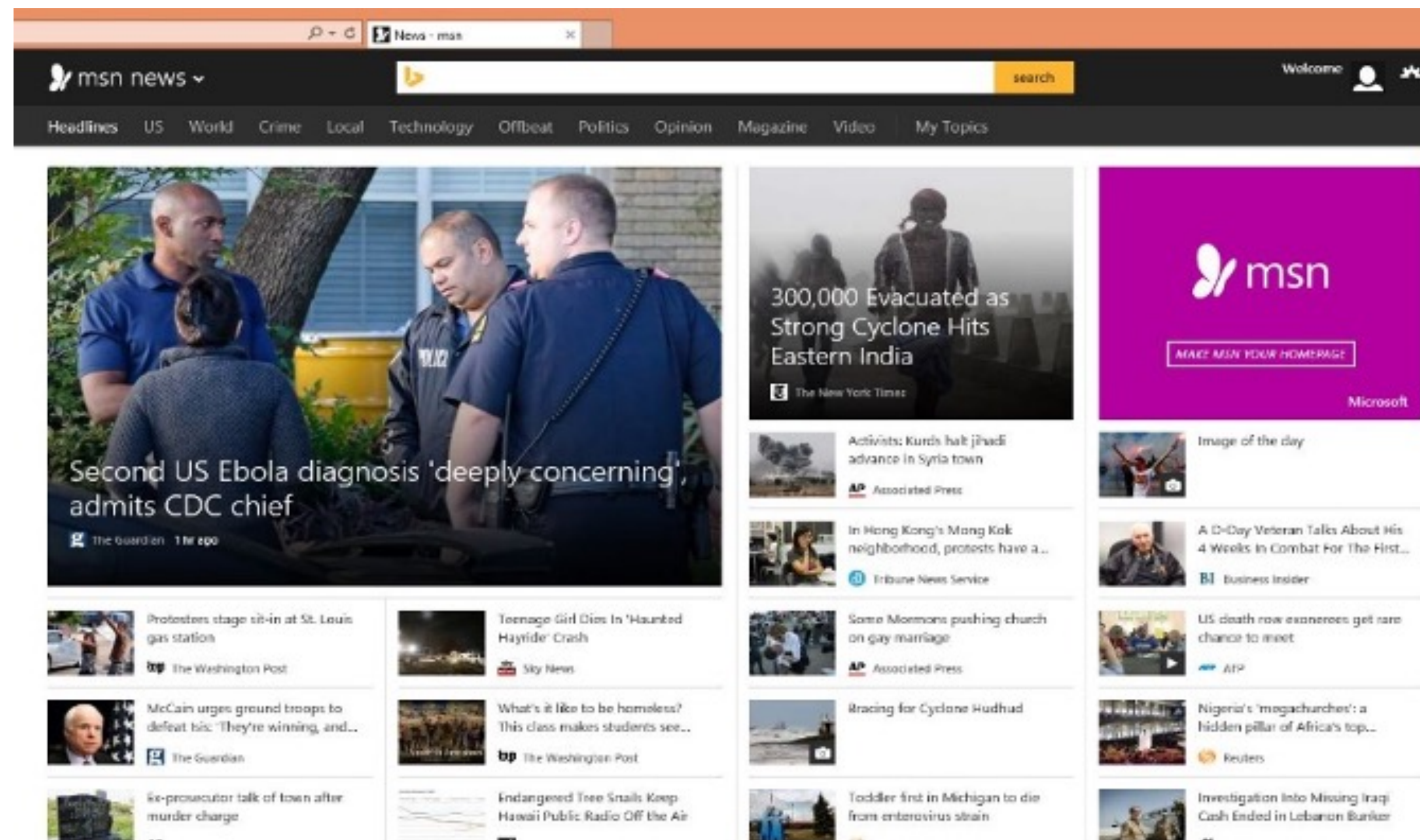
Context: user's information (e.g.,
history health conditions, age, height,
weight, job type, etc)

Decisions (arms): news articles

Goal: learn to maximizes
user click rate

Examples:

Personalize
recommendation system



Context: user's information (e.g., history health conditions, age, height, weight, job type, etc)

Decisions (arms): news articles

Goal: learn to maximizes
user click rate

Different users have different
preferences on news, so
need to personalize

Equivalently, it is an MDP with $H = 1$

Finite horizon MDP with $H = 1$

$$\mathcal{M} = \{\mathcal{X}, \mathcal{A}, r, H = 1, \mu\}$$

Equivalently, it is an MDP with $H = 1$

Finite horizon MDP with $H = 1$

$$\mathcal{M} = \{\mathcal{X}, \mathcal{A}, r, H = 1, \mu\}$$

Objective function:

$$\max_{\pi \in \Pi} \mathbb{E}_{x \sim \mu} [r(x, \pi(x))]$$

Equivalently, it is an MDP with $H = 1$

Finite horizon MDP with $H = 1$

$$\mathcal{M} = \{\mathcal{X}, \mathcal{A}, r, H = 1, \mu\}$$

Objective function:

$$\max_{\pi \in \Pi} \mathbb{E}_{x \sim \mu} [r(x, \pi(x))]$$

For simplicity, we assume reward is deterministic;
The challenge is really from randomness in contexts

The Regret Metric

Fix a policy class Π (think about π as a classifier from $x \rightarrow a$)

Denote optimal policy $\pi^\star = \arg \max_{\pi \in \Pi} \mathbb{E}_{x \sim \mu} r(x, \pi(x))$

The Regret Metric

Fix a policy class Π (think about π as a classifier from $x \rightarrow a$)

Denote optimal policy $\pi^\star = \arg \max_{\pi \in \Pi} \mathbb{E}_{x \sim \mu} r(x, \pi(x))$

Every iteration, learner has a policy $\pi^t \in \Pi$
(Recommends $a_t = \pi^t(x_t)$, receives reward $r_t := r(x_t, \pi(x_t))$)

The Regret Metric

Fix a policy class Π (think about π as a classifier from $x \rightarrow a$)

Denote optimal policy $\pi^\star = \arg \max_{\pi \in \Pi} \mathbb{E}_{x \sim \mu} r(x, \pi(x))$

Every iteration, learner has a policy $\pi^t \in \Pi$
(Recommends $a_t = \pi^t(x_t)$, receives reward $r_t := r(x_t, \pi(x_t))$)

$$\text{Regret}_T = T \mathbb{E}_{x \sim \mu} [r(x, \pi^\star(x))] - \sum_{t=0}^{T-1} \mathbb{E}_{x \sim \mu} [r(x, \pi^t(x))]$$

The Regret Metric

Fix a policy class Π (think about π as a classifier from $x \rightarrow a$)

Denote optimal policy $\pi^\star = \arg \max_{\pi \in \Pi} \mathbb{E}_{x \sim \mu} r(x, \pi(x))$

Every iteration, learner has a policy $\pi^t \in \Pi$
(Recommends $a_t = \pi^t(x_t)$, receives reward $r_t := r(x_t, \pi(x_t))$)

$$\text{Regret}_T = T \mathbb{E}_{x \sim \mu} [r(x, \pi^\star(x))] - \sum_{t=0}^{T-1} \mathbb{E}_{x \sim \mu} [r(x, \pi^t(x))]$$

Total expected reward if we always
uses $\pi^\star$ to recommend

The Regret Metric

Fix a policy class Π (think about π as a classifier from $x \rightarrow a$)

Denote optimal policy $\pi^\star = \arg \max_{\pi \in \Pi} \mathbb{E}_{x \sim \mu} r(x, \pi(x))$

Every iteration, learner has a policy $\pi^t \in \Pi$
(Recommends $a_t = \pi^t(x_t)$, receives reward $r_t := r(x_t, \pi(x_t))$)

$$\text{Regret}_T = T \mathbb{E}_{x \sim \mu} [r(x, \pi^\star(x))] - \sum_{t=0}^{T-1} \mathbb{E}_{x \sim \mu} [r(x, \pi^t(x))]$$

Total expected reward if we always
uses $\pi^\star$ to recommend

Total expected reward of our learned
sequence of policies

Outline for today:



1. Introduction of the model

2. Algorithm

3. Theory and some practical considerations

Ingredient 1: Importance Weighting

The key challenging here is that we observe $r_t := r(x_t, a_t)$,
but we do not know $r(x_t, a)$ for $a \neq a_t$

Ingredient 1: Importance Weighting

The key challenging here is that we observe $r_t := r(x_t, a_t)$,
but we do not know $r(x_t, a)$ for $a \neq a_t$

Importance weighting actually allows us to get
unbiased estimate for ALL actions!

Ingredient 1: Importance Weighting

The key challenging here is that we observe $r_t := r(x_t, a_t)$,
but we do not know $r(x_t, a)$ for $a \neq a_t$

Importance weighting actually allows us to get
unbiased estimate for ALL actions!

Assume $a_t \sim p$ ($p \in \Delta(\mathcal{A})$), and we log $p(a_t)$, receive $r_t = r(x_t, a_t)$,

Ingredient 1: Importance Weighting

The key challenging here is that we observe $r_t := r(x_t, a_t)$,
but we do not know $r(x_t, a)$ for $a \neq a_t$

Importance weighting actually allows us to get
unbiased estimate for ALL actions!

Assume $a_t \sim p$ ($p \in \Delta(\mathcal{A})$), and we log $p(a_t)$, receive $r_t = r(x_t, a_t)$,

For all $a \in \mathcal{A}$, define $\hat{\mathbf{r}}[a] = \frac{r(x_t, a)\mathbf{1}[a = a_t]}{p(a_t)}$,

Ingredient 1: Importance Weighting

The key challenge here is that we observe $r_t := r(x_t, a_t)$,
but we do not know $r(x_t, a)$ for $a \neq a_t$

Importance weighting actually allows us to get
unbiased estimate for ALL actions!

Assume $a_t \sim p$ ($p \in \Delta(\mathcal{A})$), and we log $p(a_t)$, receive $r_t = r(x_t, a_t)$,

For all $a \in \mathcal{A}$, define $\hat{\mathbf{r}}[a] = \frac{r(x_t, a)\mathbf{1}[a = a_t]}{p(a_t)}$, $\hat{\mathbf{r}} := \begin{bmatrix} 0 \\ 0 \\ \dots \\ r_t/p(a_t) \\ 0, \\ \dots \\ 0 \end{bmatrix}$

Ingredient 1: Importance Weighting

The key challenging here is that we observe $r_t := r(x_t, a_t)$,
but we do not know $r(x_t, a)$ for $a \neq a_t$

Importance weighting actually allows us to get
unbiased estimate for ALL actions!

Assume $a_t \sim p$ ($p \in \Delta(\mathcal{A})$), and we log $p(a_t)$, receive $r_t = r(x_t, a_t)$,

For all $a \in \mathcal{A}$, define $\hat{\mathbf{r}}[a] = \frac{r(x_t, a)\mathbf{1}[a = a_t]}{p(a_t)}$, $\hat{\mathbf{r}} := \begin{bmatrix} 0 \\ 0 \\ \dots \\ r_t/p(a_t) \\ 0, \\ \dots \\ 0 \end{bmatrix}$

$\mathbb{E}_{a_t \sim p} \hat{\mathbf{r}}[a] = r(x_t, a), \forall a \in \mathcal{A}$

Proving Importance Weighting

Assume $a_t \sim p$ ($p \in \Delta(\mathcal{A})$), and we log $p_t(a)$, receive $r_t = r(x_t, a_t)$,

For all $a \in \mathcal{A}$, define $\hat{r}[a] = \frac{r(x_t, a)\mathbf{1}[a = a_t]}{p(a_t)}$, we have: $\mathbb{E}_{a_t \sim p} \hat{r}[a] = r(x_t, a), \forall a \in \mathcal{A}$

Proving Importance Weighting

Assume $a_t \sim p$ ($p \in \Delta(\mathcal{A})$), and we log $p_t(a)$, receive $r_t = r(x_t, a_t)$,

For all $a \in \mathcal{A}$, define $\hat{r}[a] = \frac{r(x_t, a)\mathbf{1}[a = a_t]}{p(a_t)}$, we have: $\mathbb{E}_{a_t \sim p} \hat{r}[a] = r(x_t, a), \forall a \in \mathcal{A}$

Consider any $a \in \mathcal{A}$:

Proving Importance Weighting

Assume $a_t \sim p$ ($p \in \Delta(\mathcal{A})$), and we log $p_t(a)$, receive $r_t = r(x_t, a_t)$,

For all $a \in \mathcal{A}$, define $\hat{r}[a] = \frac{r(x_t, a)\mathbf{1}[a = a_t]}{p(a_t)}$, we have: $\mathbb{E}_{a_t \sim p} \hat{r}[a] = r(x_t, a), \forall a \in \mathcal{A}$

Consider any $a \in \mathcal{A}$:

$$\mathbb{E}_{a_t \sim p} \frac{r(x_t, a)\mathbf{1}[a = a_t]}{p(a_t)} = \sum_{a_t \in \mathcal{A}} p(a_t) \frac{r(x_t, a)\mathbf{1}[a = a_t]}{p(a_t)}$$

Proving Importance Weighting

Assume $a_t \sim p$ ($p \in \Delta(\mathcal{A})$), and we log $p_t(a)$, receive $r_t = r(x_t, a_t)$,

For all $a \in \mathcal{A}$, define $\hat{r}[a] = \frac{r(x_t, a)\mathbf{1}[a = a_t]}{p(a_t)}$, we have: $\mathbb{E}_{a_t \sim p} \hat{r}[a] = r(x_t, a), \forall a \in \mathcal{A}$

Consider any $a \in \mathcal{A}$:

$$\begin{aligned}\mathbb{E}_{a_t \sim p} \frac{r(x_t, a)\mathbf{1}[a = a_t]}{p(a_t)} &= \sum_{a_t \in \mathcal{A}} p(a_t) \frac{r(x_t, a)\mathbf{1}[a = a_t]}{p(a_t)} \\ &= p(a) \frac{r(x_t, a)}{p(a)} = r(x_t, a)\end{aligned}$$

Ingredient 2: Reward-sensitive Classification Oracle

Recall classic classification:

Ingredient 2: Reward-sensitive Classification Oracle

Recall classic classification:

$$\{x_i, y_i\}_{i=1}^N, \quad x_i \in \mathcal{X}, y_i \in \{1, \dots, |\mathcal{A}|\}$$

Ingredient 2: Reward-sensitive Classification Oracle

Recall classic classification:

$$\{x_i, y_i\}_{i=1}^N, \quad x_i \in \mathcal{X}, y_i \in \{1, \dots, |\mathcal{A}|\}$$

$$\arg \max_{\pi \in \Pi} \sum_{i=1}^N \mathbf{1}\{\pi(x_i) = y_i\}$$

Ingredient 2: Reward-sensitive Classification Oracle

Recall classic classification:

$$\{x_i, y_i\}_{i=1}^N, \quad x_i \in \mathcal{X}, y_i \in \{1, \dots, |\mathcal{A}|\}$$

$$\arg \max_{\pi \in \Pi} \sum_{i=1}^N \mathbf{1}\{\pi(x_i) = y_i\}$$

Let's generalize it to Reward-Sensitive Classification:

Ingredient 2: Reward-sensitive Classification Oracle

Recall classic classification:

$$\{x_i, y_i\}_{i=1}^N, \quad x_i \in \mathcal{X}, y_i \in \{1, \dots, |\mathcal{A}|\}$$

$$\arg \max_{\pi \in \Pi} \sum_{i=1}^N \mathbf{1}\{\pi(x_i) = y_i\}$$

Let's generalize it to Reward-Sensitive Classification:

$$\{x_i, \mathbf{r}_i\}_{i=1}^N, \quad x_i \in \mathcal{X}, \mathbf{r}_i \in [0, 1]^{|\mathcal{A}|}$$

Ingredient 2: Reward-sensitive Classification Oracle

Recall classic classification:

$$\{x_i, y_i\}_{i=1}^N, \quad x_i \in \mathcal{X}, y_i \in \{1, \dots, |\mathcal{A}|\}$$

$$\arg \max_{\pi \in \Pi} \sum_{i=1}^N \mathbf{1}\{\pi(x_i) = y_i\}$$

Let's generalize it to Reward-Sensitive Classification:

$$\{x_i, \mathbf{r}_i\}_{i=1}^N, \quad x_i \in \mathcal{X}, \mathbf{r}_i \in [0,1]^{|\mathcal{A}|} \quad \left(x, \begin{bmatrix} r[1] \\ r[2] \\ \dots \\ r[|\mathcal{A}|] \end{bmatrix} \right)$$

Ingredient 2: Reward-sensitive Classification Oracle

Recall classic classification:

$$\{x_i, y_i\}_{i=1}^N, \quad x_i \in \mathcal{X}, y_i \in \{1, \dots, |\mathcal{A}|\}$$

$$\arg \max_{\pi \in \Pi} \sum_{i=1}^N \mathbf{1}\{\pi(x_i) = y_i\}$$

Let's generalize it to Reward-Sensitive Classification:

$$\{x_i, \mathbf{r}_i\}_{i=1}^N, \quad x_i \in \mathcal{X}, \mathbf{r}_i \in [0, 1]^{|\mathcal{A}|}$$

$$\arg \max_{\pi \in \Pi} \sum_{i=1}^N \mathbf{r}_i[\pi(x_i)]$$

$$\left(x, \begin{bmatrix} r[1] \\ r[2] \\ \dots \\ r[|\mathcal{A}|] \end{bmatrix} \right)$$

Ingredient 2: Reward-sensitive Classification Oracle

Recall classic classification:

$$\{x_i, y_i\}_{i=1}^N, \quad x_i \in \mathcal{X}, y_i \in \{1, \dots, |\mathcal{A}|\}$$

$$\arg \max_{\pi \in \Pi} \sum_{i=1}^N \mathbf{1}\{\pi(x_i) = y_i\}$$

Let's generalize it to Reward-Sensitive Classification:

$$\{x_i, \mathbf{r}_i\}_{i=1}^N, \quad x_i \in \mathcal{X}, \mathbf{r}_i \in [0, 1]^{|\mathcal{A}|}$$

$$\arg \max_{\pi \in \Pi} \sum_{i=1}^N \mathbf{r}_i[\pi(x_i)]$$

$$\left(x, \begin{bmatrix} r[1] \\ r[2] \\ \dots \\ r[|\mathcal{A}|] \end{bmatrix} \right)$$

We will do reduction to RSC

Summary so far:

1. Importance Weighting: we “magically” get unbiased estimate for all actions!

Assume $a_t \sim p$ ($p \in \Delta(\mathcal{A})$), For all $a \in \mathcal{A}$, define $\hat{\mathbf{r}}[a] = \frac{r(x_t, a)\mathbf{1}[a = a_t]}{p(a_t)}$, we have:

$$\hat{\mathbf{r}} := \begin{bmatrix} 0 \\ 0 \\ \vdots \\ r_t/p(a_t) \\ 0, \\ \vdots \\ 0 \end{bmatrix}$$

$$\mathbb{E}_{a_t \sim p} \hat{\mathbf{r}}[a] = r(x_t, a), \forall a \in \mathcal{A}$$

Summary so far:

1. Importance Weighting: we “magically” get unbiased estimate for all actions!

Assume $a_t \sim p$ ($p \in \Delta(\mathcal{A})$), For all $a \in \mathcal{A}$, define $\hat{\mathbf{r}}[a] = \frac{r(x_t, a)\mathbf{1}[a = a_t]}{p(a_t)}$, we have:

$$\hat{\mathbf{r}} := \begin{bmatrix} 0 \\ 0 \\ \dots \\ r_t/p(a_t) \\ 0, \\ \dots \\ 0 \end{bmatrix}$$

$$\mathbb{E}_{a_t \sim p} \hat{\mathbf{r}}[a] = r(x_t, a), \forall a \in \mathcal{A}$$

2. Reward-Sensitive Classification:

$$\{x_i, \mathbf{r}_i\}_{i=1}^N, \quad x_i \in \mathcal{X}, \mathbf{r}_i \in [0,1]^{|\mathcal{A}|}$$

$$\arg \max_{\pi \in \Pi} \sum_{i=1}^N \mathbf{r}_i[\pi(x_i)]$$

$$\left(x, \begin{bmatrix} r[1] \\ r[2] \\ \dots \\ r[|\mathcal{A}|] \end{bmatrix} \right)$$

Put things together: Explore and Commit

For $t = 0 \rightarrow N - 1$: (# exploration phase)

Put things together: Explore and Commit

For $t = 0 \rightarrow N - 1$: (# exploration phase)

1. Observe $x_t \sim \mu$

Put things together: Explore and Commit

For $t = 0 \rightarrow N - 1$: (# exploration phase)

1. Observe $x_t \sim \mu$

2. **Uniform-randomly** sample $a_t \sim \text{Unif}(\mathcal{A})$, receive reward $r_t = r(x_t, a_t)$

Put things together: Explore and Commit

For $t = 0 \rightarrow N - 1$: (# exploration phase)

1. Observe $x_t \sim \mu$

2. **Uniform-randomly** sample $a_t \sim \text{Unif}(\mathcal{A})$, receive reward $r_t = r(x_t, a_t)$

3. Use **IW**, form unbiased estimate $\hat{r}_t[a] = \begin{cases} 0 & a \neq a_t \\ \frac{r_t}{1/|\mathcal{A}|} & a = a_t \end{cases}$

Put things together: Explore and Commit

For $t = 0 \rightarrow N - 1$: (# exploration phase)

1. Observe $x_t \sim \mu$

2. **Uniform-randomly** sample $a_t \sim \text{Unif}(\mathcal{A})$, receive reward $r_t = r(x_t, a_t)$

3. Use **IW**, form unbiased estimate $\hat{\mathbf{r}}_t[a] = \begin{cases} 0 & a \neq a_t \\ \frac{r_t}{1/|\mathcal{A}|} & a = a_t \end{cases}$

$$\hat{\mathbf{r}} := \begin{bmatrix} 0 \\ 0 \\ \dots \\ r_t/p(a_t) \\ 0, \\ \dots \\ 0 \end{bmatrix}$$

Put things together: Explore and Commit

For $t = 0 \rightarrow N - 1$: (# exploration phase)

1. Observe $x_t \sim \mu$

2. **Uniform-randomly** sample $a_t \sim \text{Unif}(\mathcal{A})$, receive reward $r_t = r(x_t, a_t)$

3. Use **IW**, form unbiased estimate $\hat{\mathbf{r}}_t[a] = \begin{cases} 0 & a \neq a_t \\ \frac{r_t}{1/|\mathcal{A}|} & a = a_t \end{cases}$

Call RSC oracle: $\hat{\pi} = \arg \max_{\pi \in \Pi} \sum_{i=0}^{N-1} \hat{\mathbf{r}}_i[\pi(x_i)]$

$$\hat{\mathbf{r}} := \begin{bmatrix} 0 \\ 0 \\ \dots \\ r_t/p(a_t) \\ 0, \\ \dots \\ 0 \end{bmatrix}$$

Put things together: Explore and Commit

For $t = 0 \rightarrow N - 1$: (# exploration phase)

1. Observe $x_t \sim \mu$

2. **Uniform-randomly** sample $a_t \sim \text{Unif}(\mathcal{A})$, receive reward $r_t = r(x_t, a_t)$

3. Use **IW**, form unbiased estimate $\hat{\mathbf{r}}_t[a] = \begin{cases} 0 & a \neq a_t \\ \frac{r_t}{1/|\mathcal{A}|} & a = a_t \end{cases}$

Call RSC oracle: $\hat{\pi} = \arg \max_{\pi \in \Pi} \sum_{i=0}^{N-1} \hat{\mathbf{r}}_i[\pi(x_i)]$

$$\hat{\mathbf{r}} := \begin{bmatrix} 0 \\ 0 \\ \dots \\ r_t/p(a_t) \\ 0, \\ \dots \\ 0 \end{bmatrix}$$

For $t = N \rightarrow T - 1$: (# exploitation phase)

Put things together: Explore and Commit

For $t = 0 \rightarrow N - 1$: (# exploration phase)

1. Observe $x_t \sim \mu$

2. **Uniform-randomly** sample $a_t \sim \text{Unif}(\mathcal{A})$, receive reward $r_t = r(x_t, a_t)$

3. Use **IW**, form unbiased estimate $\hat{\mathbf{r}}_t[a] = \begin{cases} 0 & a \neq a_t \\ \frac{r_t}{1/|\mathcal{A}|} & a = a_t \end{cases}$

Call RSC oracle: $\hat{\pi} = \arg \max_{\pi \in \Pi} \sum_{i=0}^{N-1} \hat{\mathbf{r}}_i[\pi(x_i)]$

$$\hat{\mathbf{r}} := \begin{bmatrix} 0 \\ 0 \\ \dots \\ r_t/p(a_t) \\ 0, \\ \dots \\ 0 \end{bmatrix}$$

For $t = N \rightarrow T - 1$: (# exploitation phase)

1. Observe $x_t \sim \mu$, and play $a_t = \hat{\pi}(x_t)$

Outline for today:

✓ 1. Introduction of the model

✓ 2. Algorithm

3. Theory and some practical considerations

Theory of the Explore and Commit Algorithm

For simplicity, assume Π is discrete (but could be exponential large)

Theory of the Explore and Commit Algorithm

For simplicity, assume Π is discrete (but could be exponential large)

[Theorem—informal] W/ high probability, properly setting the hyper-parameter N , Explore-and-Commit has the following regret:

$$\text{Regret}_T = T\mathbb{E}_{x \sim \mu}[r(x, \pi^\star(x))] - \sum_{t=0}^{T-1} \mathbb{E}_{x \sim \mu}[r(x, \pi^t(x))] = O\left(T^{2/3} K^{1/3} \cdot \ln(|\Pi|)^{1/3}\right)$$

Practical Consideration

Instead of setting a hard threshold for explore and commit, we often interleave explore and exploitation

Practical Consideration

Instead of setting a hard threshold for explore and commit, we often interleave explore and exploitation

ϵ -greedy:

Every iteration t :

With probability $1 - \epsilon$, we play $a_t = \pi^t(x_t)$,
and w/ probability ϵ , we play $a_t \sim \text{Unif}(\mathcal{A})$

Practical Consideration

Instead of setting a hard threshold for explore and commit, we often interleave explore and exploitation

ϵ -greedy:

Every iteration t :

With probability $1 - \epsilon$, we play $a_t = \pi^t(x_t)$,
and w/ probability ϵ , we play $a_t \sim \text{Unif}(\mathcal{A})$

Q: What's the action distribution induced by ϵ -greedy at iteration t ?

Practical Consideration

Instead of setting a hard threshold for explore and commit, we often interleave explore and exploitation

ϵ -greedy:

Every iteration t :

With probability $1 - \epsilon$, we play $a_t = \pi^t(x_t)$,
and w/ probability ϵ , we play $a_t \sim \text{Unif}(\mathcal{A})$

Q: What's the action distribution induced by ϵ -greedy at iteration t ?

$$a \sim p_t, \quad p_t = (1 - \epsilon)\delta(\pi^t(x_t)) + \epsilon\text{Unif}(\mathcal{A})$$

Put things together: greedy

For $t = 0 \rightarrow \infty$ (# interleave exploration & exploitation)

1. Observe $x_t \sim \mu$

Put things together: greedy

For $t = 0 \rightarrow \infty$ (# interleave exploration & exploitation)

1. Observe $x_t \sim \mu$

2. Use ϵ -greedy to form action distribution $p_t = (1 - \epsilon)\delta(\pi^t(x_t)) + \epsilon\text{Unif}(\mathcal{A})$

Put things together: greedy

For $t = 0 \rightarrow \infty$ (# interleave exploration & exploitation)

1. Observe $x_t \sim \mu$
2. Use ϵ -greedy to form action distribution $p_t = (1 - \epsilon)\delta(\pi^t(x_t)) + \epsilon\text{Unif}(\mathcal{A})$
3. Play $a_t \sim p_t$, and observe $r_t := r(x_t, a_t)$

Put things together: greedy

For $t = 0 \rightarrow \infty$ (# interleave exploration & exploitation)

1. Observe $x_t \sim \mu$
2. Use ϵ -greedy to form action distribution $p_t = (1 - \epsilon)\delta(\pi^t(x_t)) + \epsilon\text{Unif}(\mathcal{A})$
3. Play $a_t \sim p_t$, and observe $r_t := r(x_t, a_t)$
4. Via IW, form unbiased estimate $\hat{\mathbf{r}}_t$

Put things together: greedy

For $t = 0 \rightarrow \infty$ (# interleave exploration & exploitation)

1. Observe $x_t \sim \mu$
2. Use ϵ -greedy to form action distribution $p_t = (1 - \epsilon)\delta(\pi^t(x_t)) + \epsilon\text{Unif}(\mathcal{A})$
3. Play $a_t \sim p_t$, and observe $r_t := r(x_t, a_t)$
4. Via IW, form unbiased estimate $\hat{\mathbf{r}}_t$
5. Update via RSC oracle: $\pi^{t+1} = \arg \max_{\pi \in \Pi} \sum_{i=0}^t \hat{\mathbf{r}}_i[\pi(x_i)]$

Put things together: greedy

For $t = 0 \rightarrow \infty$ (# interleave exploration & exploitation)

1. Observe $x_t \sim \mu$

2. Use ϵ -greedy to form action distribution $p_t = (1 - \epsilon)\delta(\pi^t(x_t)) + \epsilon\text{Unif}(\mathcal{A})$

3. Play $a_t \sim p_t$, and observe $r_t := r(x_t, a_t)$

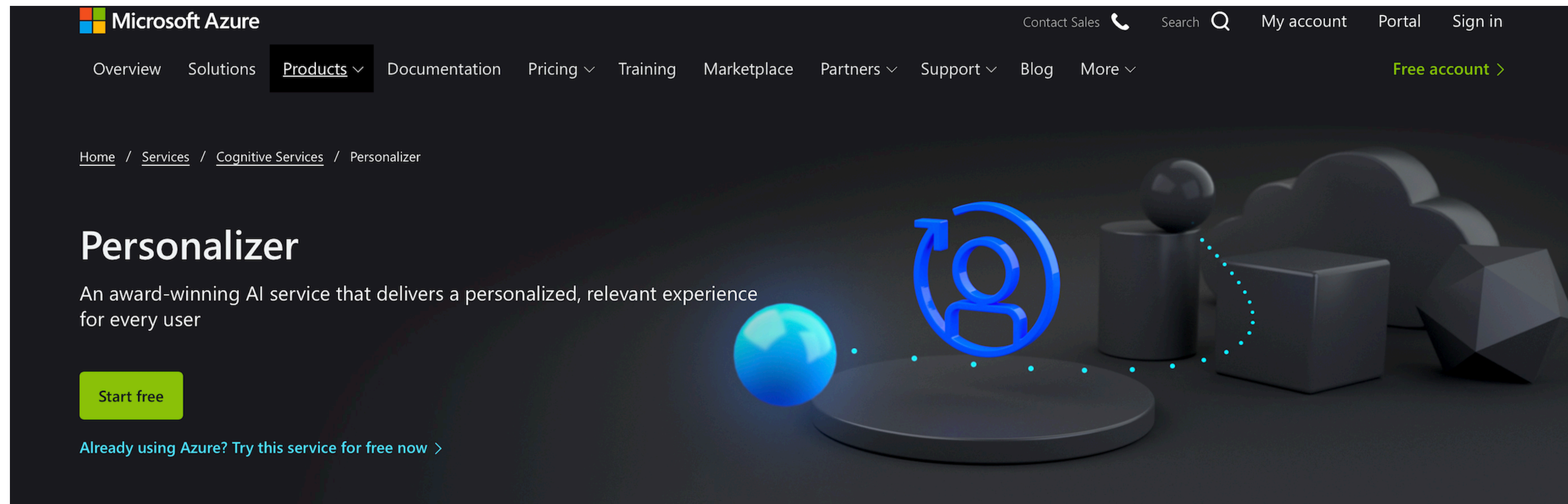
4. Via IW, form unbiased estimate $\hat{\mathbf{r}}_t$

5. Update via RSC oracle: $\pi^{t+1} = \arg \max_{\pi \in \Pi} \sum_{i=0}^t \hat{\mathbf{r}}_i[\pi(x_i)]$

(Additionally 6. Gradually decay $\epsilon \dots$)

CB algorithm is being used in real world application at Microsoft:

<https://azure.microsoft.com/en-us/services/cognitive-services/personalizer/>



The screenshot shows the Microsoft Azure website's interface for the Personalizer service. At the top, the Microsoft Azure logo is on the left, and navigation links for 'Contact Sales', 'Search', 'My account', 'Portal', and 'Sign in' are on the right. Below this, a horizontal menu includes 'Overview', 'Solutions', 'Products' (which is highlighted with a dark background), 'Documentation', 'Pricing', 'Training', 'Marketplace', 'Partners', 'Support', 'Blog', and 'More'. A 'Free account' link is positioned on the far right of this menu. The breadcrumb trail reads 'Home / Services / Cognitive Services / Personalizer'. The main heading 'Personalizer' is displayed in a large, bold font. Below it, a descriptive sentence states: 'An award-winning AI service that delivers a personalized, relevant experience for every user'. A prominent blue button labeled 'Start free' is located to the left of a large, abstract 3D graphic. This graphic features a glowing blue sphere, a blue '@' symbol with a circular arrow, and various dark grey geometric shapes like a cylinder, a cube, and a cloud. At the bottom left, a link reads 'Already using Azure? Try this service for free now >'. The entire page has a dark, modern aesthetic.

Microsoft Azure

Contact Sales Search My account Portal Sign in

Overview Solutions **Products** Documentation Pricing Training Marketplace Partners Support Blog More

Free account >

Home / Services / Cognitive Services / Personalizer

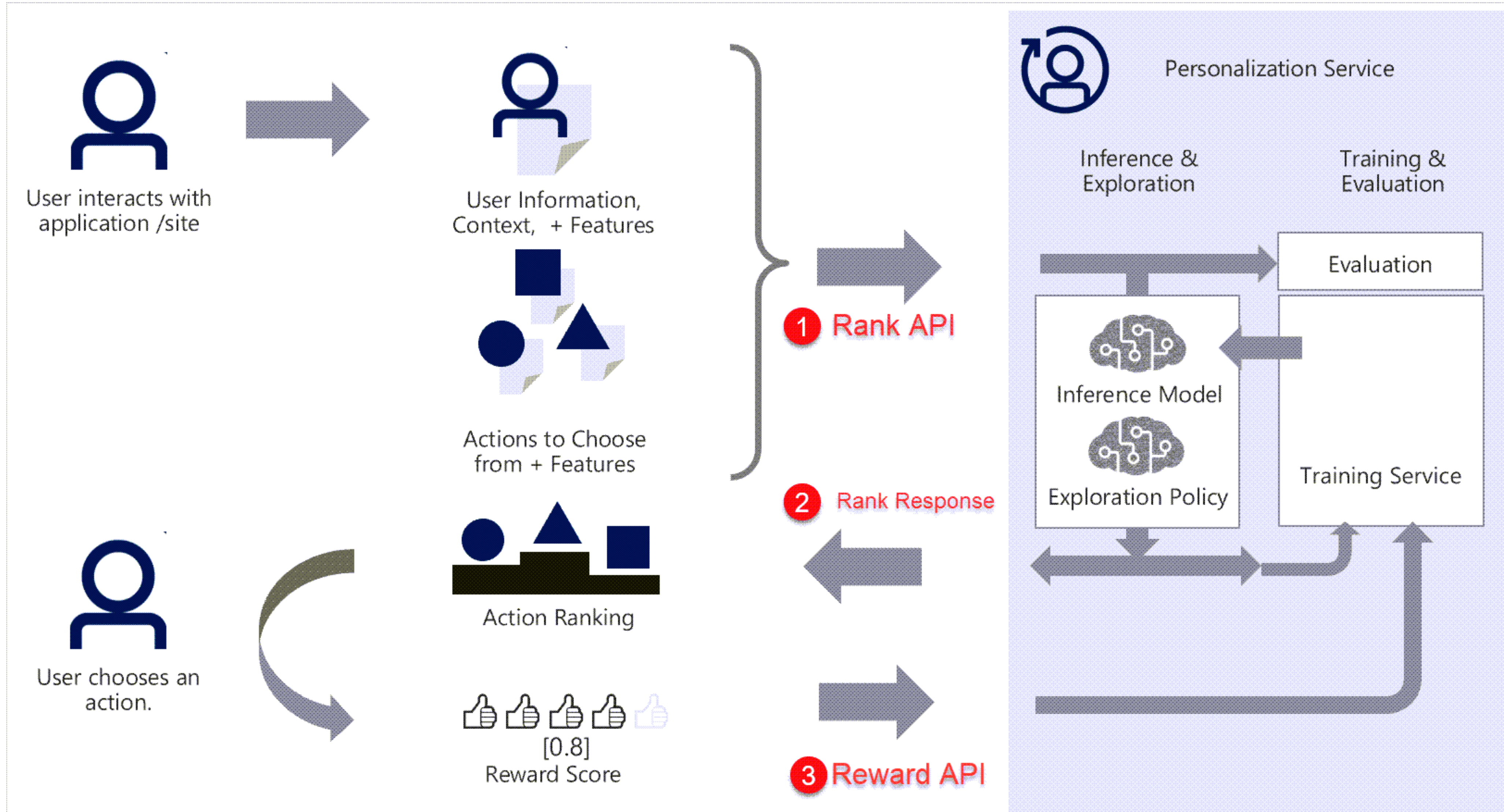
Personalizer

An award-winning AI service that delivers a personalized, relevant experience for every user

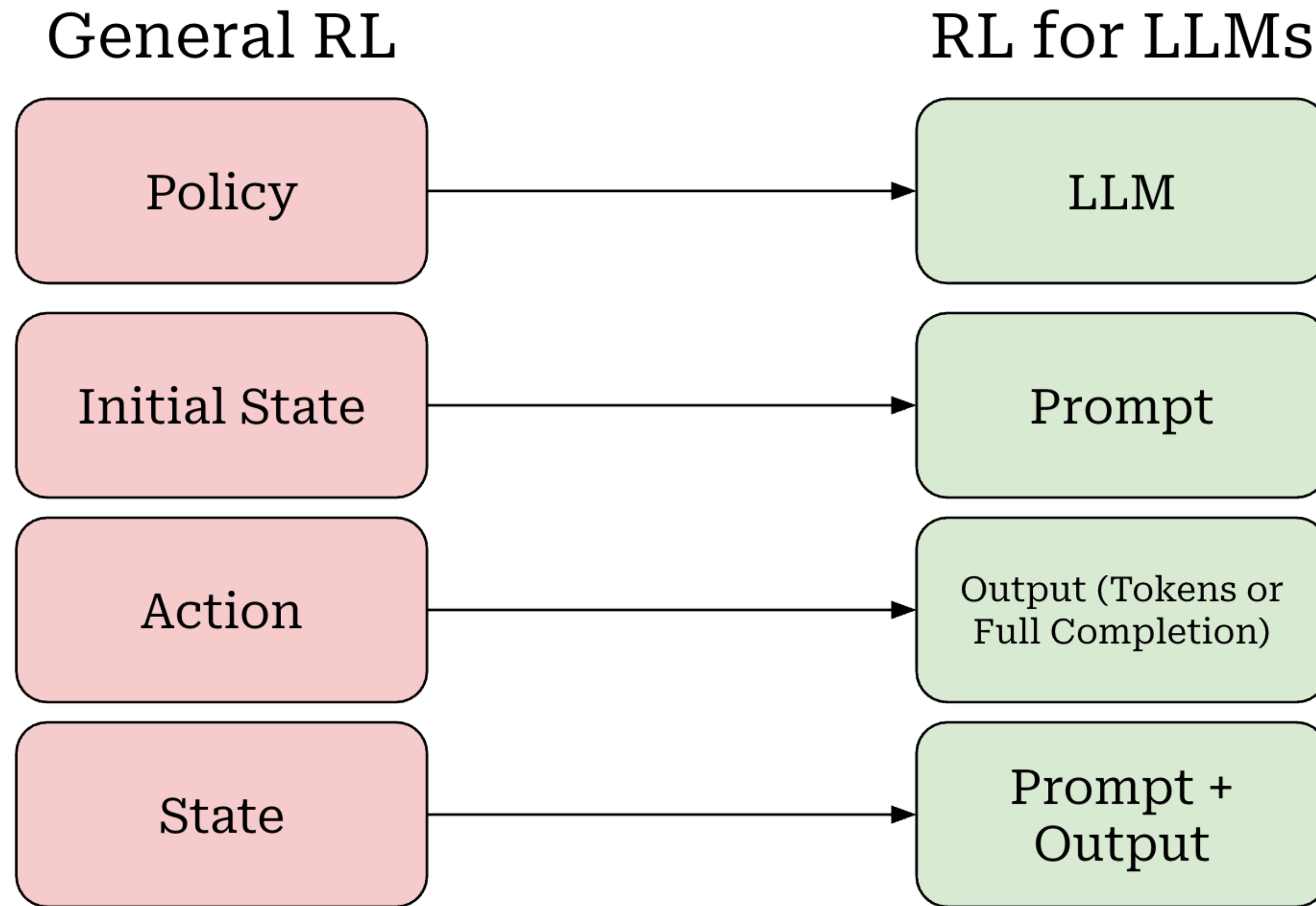
Start free

Already using Azure? Try this service for free now >

Framework



CB framework is used to train large language models



Feedback and Attendance

<https://bit.ly/4gYn2nC>

